

Karp-Rabin fingerprints

2 files check if they are the same

identify viruses on a file

idea | binary alphabet $\{0,1\}$

sequence of n bits $S = 0110 \dots 010$ $n = |S|$

$S \in [0 \dots 2^n - 1]$ cost $O(n)$ to check $S = S'$

$\hookrightarrow NS$

$F(S) = NS \mod p$ random prime number $p \in [2 \dots \gamma]$ $\gamma \ll 2^n$
 $\hookrightarrow$ user choose

issue

$F(S) \neq F(S') \Rightarrow S \neq S'$ OK

$F(S) = F(S') \rightarrow S = S'$ OK

$\rightarrow S \neq S'$ **Problem!** $\rightarrow$ unavoidable probability

Error

$k_1 \mapsto NS$ $k_1 \neq k_2 \wedge (k_1 \mod p = k_2 \mod p)$
 $k_2 \mapsto NS'$

$\hookrightarrow$ Before compute

choose γ sufficient large

choose uniformly and randomly a prime in $[2 \dots \gamma]$

p is bad prime if $k_1 \mod p = k_2 \mod p$

$37 \mod 3 = 1$

$37 \mod 5 = 2$

$40 \mod 3 = 1$

$40 \mod 5 = 0$

Bad

Good

$\Pr(\text{error}) = \Pr(k_1 \neq k_2 \wedge (k_1 \mod p = k_2 \mod p))$

$= \frac{\# \text{ Bad primes}}{\# \text{ primes in } [2 \dots \gamma]}$

$\gamma / \ln \gamma$

$\hookrightarrow$ density of primes

Fact | A number with n bits has $\leq n$ distinct prime dividing X

$$X = p_1^{e_1} \cdot p_2^{e_2} \cdot \dots \cdot p_r^{e_r} \quad 2025 = 3^4 \cdot 5^2$$

$$p_i \geq 2 \Rightarrow e_1 + e_2 + \dots + e_r \leq n$$

$$X \geq 2^{e_1 + \dots + e_r} > 2^n \text{ no possible}$$

$$e_1 + \dots + e_r \leq n \Rightarrow r = \# \text{ distinct prime dividing } X \leq n$$

$$\Pr(\text{error}) \leq \frac{n}{\gamma / \ln \gamma} \quad \text{high probability} = \frac{1}{n^c} \rightarrow \text{constant}$$

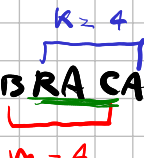
$$\text{to fix } \gamma \quad \frac{n}{\gamma / \ln \gamma} \leq \frac{1}{n^c} \Rightarrow \gamma \approx n^{c+1} \ln n$$

What if $k_1 \gamma \cdot p = k_2 \gamma \cdot p$ but p not divide $k_1 \gamma$?

Same argument $(k_1 - k_2)$

Application | Rolling hashing

Text-string $O(n)$

ABRACADABRA


$$F(\text{BRAC}) = B \cdot 131^3 + R \cdot 131^2 + A \cdot 131 + C \cdot 131^0 \pmod{p}$$

• overlap part

$$F(\text{RACA}) = R \cdot 131^3 + A \cdot 131^2 + C \cdot 131 + A \cdot 131^0 \pmod{p}$$

$$F(\text{RACA}) =$$

$$(F(\text{BRAC}) - B \cdot 131^3) \cdot 131 + A \pmod{p}$$

$$\downarrow$$

$$O(1)$$

STRING MATCHING

$P = \text{ABRA} = \text{pattern string} \quad |P| = m$

$T = \text{text string} = \text{ABRACADABRA} \quad |T| = n$

For $i = 1 \dots n - m + 1$:

(compare P with $T[i \dots i + m - 1]$)

$\rightarrow O(n \cdot m)$

KR-algorithm

$p = F(P)$

$t = F(T[1 \dots n])$

for $i = 1 \dots n - m + 1$:

compare p with t

update t in $O(1) \rightarrow t = F(T[i+1 \dots i+m])$

$$x_i = \begin{cases} 1 & p = t \text{ but } P \neq T[i \dots i+m-1] \\ 0 & \text{error} \end{cases}$$

$$\Pr[X_i] = \frac{m}{\gamma \ln \gamma}$$

$\sum_{i=1}^{n-m+1} x_i > 0 \Rightarrow$ if there is an error at some point

$$\Pr(\text{error KR-algo}) \leq \frac{n m}{\gamma \ln \gamma} \leq \frac{1}{n^c}$$

$$m \leq n \Rightarrow m \cdot n \leq n^2$$

$$\gamma \sim n^{c/2} \ln n$$

Monte Carlo algorithm

- running time is guaranteed to be $O(n+m)$
- 1-side error

Las Vegas algorithm

- running time is expected but
- no errors

KR-algo : Las Vegas version

if ($p = t$) then if ($p = T[1 \dots i+m-1]$) then return ok

$$O(n+m + Pr(KR\text{-algo}) \cdot m)$$

expected running time is $O(n+m + E[X] \cdot m)$

$$\leq \frac{n^2}{n/\ln n} \cdot m$$

$$O(1)$$